

리눅스 시스템 프로그래밍 학습 기록

남궁명수

[6] Go Concurrency: 고루틴과 스케줄링	3
[01] 고루틴의 경량성 증명: 10 만 개의 실행 흐름을 실행할 때의 메모리 소비량 분석.....	3
[01-1] OS 스레드와 고루틴의 메모리 점유 방식은 어떻게 다르며, 왜 고루틴은 수만 개를 띄워도 괜찮은가.....	3
[01-2] 고루틴 10 만 개를 생성하기 전과 후의 메모리 상태를 측정.....	3
[01-3] 이론.....	5
[01-4] 추가.....	6
[02] M:N 스케줄링의 실체: 고루틴 개수와 실제 OS 스레드(LWP) 개수의 상관관계.....	7
[02-1] 100 개의 고루틴이 들어갈 때, 리눅스 커널이 관리하는 스레드는 몇 개인가?	7
[02-2] 고루틴을 대량으로 생성하고, 프로그램이 실행되는 동안 실제 OS 스레드 개수를 측정.....	7
[02-3] 이론.....	10
[02-4] 추가.....	12
[7] Go Concurrency: 채널과 통신(Channel & CSP 모델).....	13
[01] Unbuffered Channel: 동기적 데이터 교환과 고루틴 블로킹 메커니즘 분석	13
[01-1] 100 개의 고루틴이 들어갈 때, 리눅스 커널이 관리하는 스레드는 몇 개인가?	13
[01-2] 송신자와 수신자의 속도 차이를 의도적으로 만들어, 채널이 어떻게 두 고루틴의 발거움을 맞추는지 관찰.....	13
[01-3] 이론.....	15
[01-4] 추가.....	16
[02] Unbuffered Channel: 동기적 데이터 교환과 고루틴 블로킹 메커니즘 분석	18
[02-1] 100 개의 고루틴이 들어갈 때, 리눅스 커널이 관리하는 스레드는 몇 개인가?	18
[02-2] 송신자와 수신자의 속도 차이를 의도적으로 만들어, 채널이 어떻게 두 고루틴의 발거움을 맞추는지 관찰.....	18
[02-3] 이론.....	20
[02-4] 추가.....	21
[8] Go Concurrency: 데이터 경쟁(Race Condition) 자동 탐지.....	23
[01] Go Race Detector: 런타임 분석을 통한 데이터 경쟁 자동 탐지 및 분석.....	23
[01-1] C 언어에서 육안으로 찾기 힘들었던 데이터 오염을 Go 는 어떻게 도구 레벨에서 찾아내는가?	23
[01-2] 의도적으로 데이터 레이스가 발생하는 코드를 작성([3]pthread 편의 [02]).....	23
[01-3] 이론.....	26
[01-4] 추가.....	28
[02] Mutex vs Channel: 어떤 상황에 어떤 동기화 도구를 쓸 것인가?.....	29

[02-1] 공유 메모리 방식(Mutex)과 메시지 패싱 방식(Channel)의 성능 및 코드 가독성 차이는?	29
[02-2] 두 가지 방식으로 동일한 카운팅 작업을 수행하여 결과를 비교.....	29
[02-3] 이론	32
[02-4] 추가	34
[9] Go Concurrency: 고성능 네트워크.....	35
[01] net 패키지 해부: C 소켓 프로그래밍과의 추상화 레벨 비교.....	35
[01-1] C 언어에서 육안으로 찾기 힘들었던 데이터 오염을 Go 는 어떻게 도구 레벨에서 찾아내는가?	35
[01-2] 가장 기초적인 TCP 에코 서버를 Go 로 작성.....	35
[01-3] 이론	37
[01-4] 추가	38
[02] net 패키지 해부: C 소켓 프로그래밍과의 추상화 레벨 비교.....	40
[02-1] C 언어에서 육안으로 찾기 힘들었던 데이터 오염을 Go 는 어떻게 도구 레벨에서 찾아내는가?	40
[02-2] 가장 기초적인 TCP 에코 서버를 Go 로 작성.....	40

[6] Go Concurrency: 고루틴과 스케줄링

[01] 고루틴의 경량성 증명: 10 만 개의 실행 흐름을 실행할 때의 메모리 소비량 분석

[01-1] OS 스레드와 고루틴의 메모리 점유 방식은 어떻게 다르며, 왜 고루틴은 수만 개를 띄워도 괜찮은가

- 메모리 점유의 차이: C 언어에서 다뤘던 OS 스레드(pthread)와 Go 의 고루틴은 메모리를 점유하는 방식이 어떻게 다른가?
- 안정성: 왜 고루틴은 일반적인 OS 스레드라면 시스템이 뻘었을 ‘수만 개’의 단위를 생성해도 메모리 측면에서 안전한가?

[01-2] 고루틴 10 만 개를 생성하기 전과 후의 메모리 상태를 측정

01_goroutine_mem.go

```
package main

// runtime: Go 프로그램이 실행되는 동안 메모리 관리, 가비지 컬렉션, 고루틴 스케줄링 등 핵심 시스템 인프라와 상호작용할 수 있도록 돕는 표준 라이브러리
import (
    "fmt"
    "runtime"
    "time"
)

func main(){
    // 1. 실험 전 메모리 상태 측정
    var m runtime.MemStats
    runtime.ReadMemStats(&m)
    beforeAlloc := m.Alloc

    fmt.Printf("[실험 전] 메모리 사용량: %.2f MB\n", float64(beforeAlloc)/1024/1024)

    numGoroutines := 100000

    // 2. 고루틴 10만개 생성
    for i := 0; i < numGoroutines; i++){
        go func(){
            // 고루틴이 즉시 종료되지 않도록 대기
            time.Sleep(10 * time.Second)
        }()
    }

    // 고루틴 생성 완료 후 잠시 대기(스케줄링 시간 확보)
    time.Sleep(500 * time.Millisecond)

    // 3. 실험 후 메모리 상태 측정
    runtime.ReadMemStats(&m)
    afterAlloc := m.Alloc
```

```
totalUsage := afterAlloc - beforeAlloc
perGoroutine := float64(totalUsage) / float64(numGoroutines)

fmt.Println("Wn --- 10만 개 고루틴 생성 완료 ---")
fmt.Printf("[실험 후] 메모리 사용량: %.2f MBWn", float64(afterAlloc)/1024/1024)
fmt.Printf("[결과] 증가한 총 메모리: %.2f MBWn", float64(totalUsage)/1024/1024)
fmt.Printf("[결과] 고루틴 1개당 평균 점유량: 약 %.2f KBWn", float64(perGoroutine)/1024)

fmt.Println("Wn 다른 터미널에서 top이나 htop로 실제 프로세스 점유율 확인하기")
time.Sleep(15 * time.Second) // 모니터링 시간 확보
}
```

결과:

표준 출력 결과는 다음과 같다.

```
[실험 전] 메모리 사용량: 0.05 MB

--- 10만 개 고루틴 생성 완료 ---
[실험 후] 메모리 사용량: 58.58 MB
[결과] 증가한 총 메모리: 58.54 MB
[결과] 고루틴 1개당 평균 점유량: 약 0.60 KB

다른 터미널에서 top 이나 htop 로 실제 프로세스 점유율 확인하기
```

결과 해석:

학습 결과 Go 런타임은 10만 개의 고루틴을 생성했음에도 비교적 적은 메모리만 사용한다는 것을 확인할 수 있었다.

학습 전 메모리 사용량은 약 0.05MB 였으며, 10만 개의 고루틴 생성 이후 약 58.58MB 수준으로 증가하였다. 즉 전체적으로 증가한 메모리는 약 58MB 정도였다.

이를 고루틴 개수로 나누어 계산하면, 고루틴 1개당 평균 메모리 사용량은 약 0.6KB 수준으로 측정되었다.

이는 일반적인 운영체제 스레드와 비교했을 때 매우 작은 비용이다.

리눅스의 일반 pthread는 기본적으로 수 MB 단위의 스택 공간을 예약하는 경우가 많기 때문에, 수만 개 이상의 스레드를 생성하면 메모리 사용량이 급격히 폭증한다.

반면, Go의 고루틴은 매우 작은 초기 스택(약 2KB 수준)에서 시작하고, 실제 사용량에 따라 런타임이 스택 크기를 동적으로 확장하는 구조를 사용한다.

즉, Go는 “필요한 만큼만 메모리를 사용하는 경량 실행 단위”를 제공하기 때문에 대규모 동시성 환경에서도 높은 효율성을 유지할 수 있다.

또한 top 또는 htop 명령어 시스템 레벨에서 확인했을 때도 프로세스의 RES(실제 물리 메모리 사용량)가 급격하게 폭증하지 않고 안정적으로 유지되는 것을 확인할 수 있었다.

이는 고루틴이 운영체제 스레드와 1:1로 매핑되지 않고 Go 런타임 내부의 스케줄러(M:N Scheduler)에 의해 소수의 OS 스레드 위에서 효율적으로 분산 실행되기 때문이다.

결과적으로 이번 학습을 통해 다음과 같은 사실을 확인할 수 있었다.

- 고루틴은 매우 적은 메모리 비용으로 생성 가능하다.
- Go 런타임은 스택 메모리를 동적으로 관리하여 메모리 효율을 극대화한다.
- 대량의 동시 작업은 운영체제에 직접 부담시키지 않고 런타임 수준에서 스케줄링한다.

[01-3] 이론

OS 스레드와 고루틴의 가장 큰 차이점 중 하나는 바로 스택(Stack) 메모리 관리 방식이다. 리눅스의 일반적인 OS 스레드(pthread)는 생성되는 순간 커널로부터 고정된 크기의 스택 공간을 할당받는다. 환경에 따라 차이는 있지만 보통 1MB~8MB 수준의 스택이 예약된다.

명령어: `ulimit -s` 결과: 8192

문제는 스레드 내부에서 실제로 얼마나 적은 작업을 수행하든 관계없이, 생성 시점에 이미 MB 단위 메모리가 확보된다는 점이다.

예를 들어, 단순히 네트워크 연결 하나를 기다리는 스레드라고 하더라도 다음과 같은 메모리 구조를 가진다.

[OS Thread]

- 고정 스택 8MB 예약
- 실제 사용량은 몇 KB 수준
- 하지만 남은 공간도 계속 점유 상태

즉, 대부분의 메모리가 실제로 사용되지 않더라도 “예약 상태”로 남아 있게 된다. 그래서 만약 스레드를 10만 개 생성한다고 가정하면 계산상 다음과 같은 메모리가 필요해진다.

$100,000 \times 8\text{MB} = \text{약 } 800\text{GB}$

이는 일반적인 PC나 서버 환경에서는 사실상 불가능한 수준이다.

결국 전통적인 스레드 기반 서버는 수만 개 이상의 동시 연결 처리에서 구조적인 한계에 부딪히게 된다.

반면 Go의 고루틴은 완전히 다른 접근 방식을 사용한다.

고루틴은 생성 시 단 2KB 정도의 매우 작은 스택만 할당받는다.

[Goroutine]

초기 스택 크기:

약 2KB

그리고 실행 도중 함수 호출이 깊어지거나 지역 변수 사용량이 증가하여 더 많은 스택 공간이 필요해지면, Go 런타임이 자동으로 더 큰 메모리 공간을 확보하여 스택 크기를 동적으로 확장한다.

이를 Growable Stack(확장 가능한 스택)구조라고 한다.
개념적으로는 다음과 같은 흐름이다.

2KB → 4KB → 8KB → 16KB ...

즉, “처음부터 크게 예약”하는 방식이 아니라, 실제 사용량에 맞춰 필요할 때만 확장하는 구조이다. 또한, 작업이 끝나면 사용하지 않는 스택 공간을 다시 축소하거나 회수할 수도 있기 때문에 메모리 낭비가 훨씬 적다.

이 구조 덕분에 Go는 다음과 같은 특징을 가진다.

- 수십만 개의 동시 작업 생성 가능
- 매우 낮은 메모리 오버헤드
- 대규모 네트워크 서버 구현 가능
- 클라우드/분산 시스템에 최적화
- I/O 대기 작업 처리 효율 극대화

결국 고루틴의 핵심은 단순히 “가벼운 스레드”가 아니라 “필요한 만큼만 메모리를 사용하는 실행 단위”라는 점에 있다.

그리고 이러한 구조는 현대적인 서버 환경에서 매우 강력한 장점이 된다.

실제 대규모 웹 서버, 채팅 서버, 스트리밍 시스템, 보안 에이전트, 클라우드 오케스트레이션 시스템 등이 수십 만개 동시 연결을 처리할 수 있는 이유도 결국 이러한 런타임 수준의 메모리 최적화 덕분이다.

[01-4] 추가

저사양/고효율 엔진: 보안 솔루션은 사용자 시스템의 자원을 최소한으로 쓰는 것이 핵심이다. 고루틴의 이러한 경량성은 수만 개의 파일 변경 이벤트를 동시에 감시하거나 대규모 로그를 비동기로 전송할 때 사용자가 느끼지 못할 만큼 가벼운 보안 프로그램을 만드는데 결정적인 기술이 된다.

[02] M:N 스케줄링의 실체: 고루틴 개수와 실제 OS 스레드(LWP) 개수의 상관관계

[02-1] 100 개의 고루틴이 들어갈 때, 리눅스 커널이 관리하는 스레드는 몇 개인가?

- 스레드 폭증 여부: 수백, 수천 개의 고루틴을 생성했을 때, 리눅스 커널도 그만큼의 스레드(LWP)를 생성하여 관리하는가?
- 스케줄링 주제: 고루틴의 실행 순서와 배치는 리눅스 커널이 결정하는가, 아니면 Go 런타임이 결정하는가?

[02-2] 고루틴을 대량으로 생성하고, 프로그램이 실행되는 동안 실제 OS 스레드 개수를 측정

02_mn_scheduling.go

```
package main

import (
    "fmt"
    "runtime"
    "time"
)

func main(){
    // 현재 PC의 논리 프로세서(코어) 개수 확인
    cpuCores := runtime.NumCPU()
    fmt.Printf("[환경] 시스템 논리 코어 수: %d\n", cpuCores)

    // 고루틴을 실행할 최대 OS 스레드 수 설정(기본값은 코어 수)
    // runtime.GOMAXPROCS(2), 필요시 강제로 제한 가능
    numGoroutines := 1000 // 1,000개의 고루틴 생성

    for i:= 0; i < numGoroutines; i++){
        go func(id int){
            for {
                // 무한 루프를 돌며 CPU를 소량 점유
                _ = id * id
                time.Sleep(time.Millisecond)
            }
        }(i)
    }

    fmt.Printf("[수행] %d개의 고루틴이 실행 중입니다...\n", numGoroutines)
    fmt.Println("[명령] 다른 터미널에서 다음 명령어를 입력해 실제 스레드 수(LWP)를
```

확인하세요.")

```
fmt.Println("명령어: ps -eLf | grep main | wc -l")
```

```
}
```

결과:

표준 출력은 다음과 같다.

[환경] 시스템 논리 코어 수: 2

[수행] 1000 개의 고루틴이 실행 중입니다...

[명령] 다른 터미널에서 다음 명령어를 입력해 실제 스레드 수(LWP)를 확인하세요.

명령어: ps -eLf | grep main | wc -l

```
$ ps -eLf | grep main | wc -l
```

```
12
```

결과 해석:

학습 결과, 1,000 개의 고루틴이 동시에 실행되고 있음에도 불구하고, 실제 운영체제 수준에서 생성된 스레드(LWP)의 개수는 약 1015 개 수준에 불과한 것을 확인할 수 있었다.

즉, Go 프로그램 내부에서는 1,000 개의 동시 작업이 실행 중이지만, 리눅스 커널 입장에서는 단지 소수의 스레드만 존재하는 것처럼 보인다는 의미이다.

이는 Go 런타임이 사용하는 M:N 스케줄링 구조 때문이다.

전통적인 C 언어의 pthread 모델은 일반적으로 다음과 같은 구조를 가진다.

1 개의 사용자 스레드 = 1 개의 OS 스레드

따라서 pthread 를 1,000 개 생성하면 커널 역시 1,000 개의 실제 스레드를 관리해야 한다.

이 경우 다음과 같은 비용이 발생한다.

- 스레드 생성 비용 증가
- 컨텍스트 스위칭 증가
- 커널 스케줄링 부담 증가
- 스택 메모리 사용량 폭증
- 캐시 효율 저하

결국 스레드 수가 많아질수록 운영체제 자체가 과부하 상태에 가까워진다.

반면, Go 는 고루틴을 직접 커널 스레드에 연결하지 않는다.

Go 런타임은 내부적으로 다음과 같은 구조를 사용한다.

수천 개의 고루틴(G)



Go 런타임 스케줄러



소수의 OS 스레드(M)



CPU 코어(P)

즉, 수많은 고루틴을 소수의 실제 스레드 위에서 번갈아 실행시키는 방식이다.

학습 환경에서는 논리 코어 수가 2 개였기 때문에, Go 런타임은 실제로 매우 적은 수의 OS 스레드만 유지하면서, 1,000 개의 고루틴을 효율적으로 분산 실행하였다.

특히 고루틴 내부에서 `time.Sleep()` 같은 블로킹 성격의 작업이 발생하면, Go 런타임은 해당 고루틴만 잠시 멈추고 다른 고루틴에게 CPU 실행 기회를 넘겨준다.

즉, 운영체제 커널이 아니라, Go 런타임 자체가 사용자 레벨 스케줄링(User-level Scheduling)을 수행하는 것이다.

결과적으로, 이번 학습을 통해 다음과 같은 사실을 확인할 수 있었다.

- 고루틴은 OS 스레드와 1:1 대응되지 않는다.
- Go 런타임이 수많은 고루틴을 소수의 스레드 위에서 스케줄링한다.
- 따라서 매우 적은 시스템 자원으로 대규모 동시성을 구현할 수 있다.
- 컨텍스트 스위칭 비용과 메모리 사용량이 크게 감소한다.
- 이것이 Go 가 대규모 네트워크 서버와 클라우드 환경에서 강력한 이유이다.

즉, Go 의 핵심 경쟁력은 단순히 “가벼운 스레드”가 아니라, “동시성 관리 자체를 커널이 아니라 런타임이 대신 최적화한다”는 점에 있다.

[02-3] 이론

Go 의 동시성 모델은 전통적인 운영체제 스레드 기반 모델과 매우 다르게 동작한다. 일반적인 C 언어의 pthread 는 “사용자 스레드 하나 = 커널 스레드 하나” 형태의 1:1 구조를 사용한다. 따라서 스레드를 생성할 때마다 리눅스 커널이 직접 새롭게 실행 단위를 만들어 관리해야 하며, 스레드 수가 증가할수록 커널의 스케줄링 부담도 함께 커진다.

반면, Go 는 커널 스레드를 직접 많이 생성하지 않고, Go 런타임 내부에서 자체 스케줄러를 이용해 수많은 고루틴을 효율적으로 분배한다.
이를 G-M-P 스케줄링 모델이라고 한다.

먼저 G(Goroutine)는 개발자가 go 키워드를 사용해 생성하는 실제 작업 단위이다.

```
go myFunction()
```

이렇게 생성된 고루틴은 실행할 함수 정보, 현재 실행 위치(Program Counter), 스택(Stack) 등의 상태 정보를 가진다.

고루틴은 매우 가볍고 초기 스택도 매우 작기 때문에 수십만 개까지 생성할 수 있다.

하지만 고루틴 자체가 직접 CPU 에서 실행되는 것은 아니다.
실제로 CPU 위에서 실행되는 것은 M(Machine)이라고 불리는 OS 스레드이다.

M 은 리눅스 커널이 관리하는 실제 스레드(LWP)이며, ps -eLf 같은 명령어로 확인 가능한 존재이다. 결국 CPU 는 M 만 실행할 수 있고, 고루틴은 G 는 반드시 어떤 M 위에 올라타야 실행된다.

그런데 여기서 중요한 중간 계층이 하나 더 존재한다. 바로 P(Processor)이다. P 는 단순히 CPU 자체가 아니라, “고루틴 실행 권한과 스케줄링 자원”에 가깝다.

Go 에서는 M 이 아무리 많아도 P 를 획득하지 못하면 고루틴을 실행할 수 없다.
개념적으로는 다음과 같은 구조이다.

```
G (고루틴)
↓
P (실행 권한)
↓
M (OS 스레드)
↓
CPU
```

보통 P 의 개수는 시스템의 논리 코어 개수와 동일하게 설정된다.
예를 들어, CPU 코어가 2 개라면 기본적으로 P 도 2 개가 생성된다.

```
runtime.NumCPU()
```

즉, 동시에 실제 병렬 실행 가능한 고루틴 수도 결국 P 개수에 의해 제한된다.

Go 런타임은 수천 개의 G를 소수의 M 위에서 매우 빠르게 교체하며 실행한다.
이 과정에서 핵심적인 장점이 발생한다.

전통적인 pthread 환경에서는 스레드 전환(Context Switching)이 발생할 때마다 커널이 개입한다.

예를 들어,

Thread A → Kernel Mode → Thread B

이 과정에서는 다음과 같은 작업이 필요하다.

- 레지스터 저장
- 스택 포인트 변경
- 메모리 맵 전환
- CPU 캐시 무효화 가능성
- 스케줄러 실행

즉, 커널 모드 진입 비용 자체가 상당히 비싸다.

하지만 Go는 대부분의 고루틴 전환을 커널이 아니라 런타임 내부에서 처리한다.

G1 → G2 → G3

(유저 모드 내부 교체)

리눅스 커널 입장에서는 여전히 같은 M(OS 스레드)이 계속 실행 중인 것으로만 보인다.

즉, 내부적으로 어떤 고루틴이 실행 중인지 커널은 알지 못한다.

이러한 구조를 User-level Scheduling(유저 레벨 스케줄링)이라고 한다.

결과적으로 Go는 다음과 같은 장점을 얻게 된다.

- 컨텍스트 스위칭 비용 감소
- 커널 개입 최소화
- 메모리 사용량 절감
- 대규모 동시성 처리 가능
- 높은 CPU 활용 효율
- 수십만 개 연결 처리 가능

그래서 Go는 다음과 같은 환경에서 특히 강력하다.

- 대규모 네트워크 서버
- 클라우드 인프라
- API Gateway
- 실시간 스트리밍 서버
- 보안 에이전트
- 채팅 서버
- 분산 시스템

운영체제에게 모든 스케줄링을 맡기지 말고, 런타임이 직접 동시성을 관리한다.

이것이 현대 Go가 현대적인 고성능 서버 환경에서 강력한 이유이다.

[02-4] 추가

(1) 고루틴의 단점 및 한계

디버깅 및 프로파일링의 어려움

수만 개의 고루틴이 동시에 실행될 수 있기 때문에 문제가 발생했을 때 추적하기 어렵다. 특정 고루틴에서 교착상태(Deadlock)나 메모리 누수가 발생할 경우, 어떤 고루틴이 문제를 일으켰는지 정확히 파악하려면 pprof 같은 프로파일링 도구에 크게 의존해야 한다.

가비지 컬렉터(GC) 부하 및 메모리 누수

고루틴이 종료되지 않고 백그라운드에서 계속 실행되거나(고루틴 누수), 너무 많은 고루틴을 생성하면 메모리 사용량이 급증한다. Go의 가비지 컬렉터는 이 많은 고루틴과 힙 메모리를 주기적으로 스캔해야 하므로, 고루틴을 무분별하게 생성하면 오히려 GC로 인한 Stop-The-Worlds(STW) 지연이 발생해 성능이 크게 저하될 수 있다.

컨텍스트 스위칭 비용(특정 상황)

고루틴 간의 전환(Context Switching)은 OS 스레드 전환보다 훨씬 빠르다. 하지만, I/O 작업이 아닌 CPU 연산만 과도하게 수행하는 고루틴들이 수십만개 생성되면 Go 런타임 스케줄러가 이를 공평하게 처리하기 위해 지속적으로 스위칭을 시도하게 된다. 이로인해 불필요한 오버헤드가 발생하여 성능이 오히려 떨어질 수 있다.

잘못된 채널 사용으로 인한 교착상태(Deadlock)

고루틴 간의 통신은 주로 Channel을 통해 이루어지는데, 데이터를 보내거나 받을 때 블로킹(Blocking)이 일어난다.

이때 상대방 고루틴이 응답하지 않거나 대기열이 꼬이게 되면 전체 프로세스가 멈추는 데드락에 빠지기 매우 쉽다.

동기화 문제(Race Condition)

고루틴은 메모리 공간을 공유하기 때문에 여러 고루틴이 동시에 같은 변수에 접근하여 값을 변경하려고 하면 경쟁 상태가 발생한다. 이를 방지하려면 Mutex 등을 사용해 락을 걸어야 하는데, 이 과정에서 코드가 복잡해지고 성능 하락이 발생할 수 있다.

(2) 탐지 회피와 가시성

악성코드가 Go로 작성된 경우, 시스템 모니터링 도구(기존 백신 등)에서 “스레드 개수”만 감시하면 해당 프로세스가 얼마나 많은 병렬 작업을 수행하는지 정확히 파악하기 어렵다.

(3) 보안 엔진 최적화

안랩의 실시간 감시 엔진이 수천 개의 시스템 이벤트를 동시에 처리할 때, OS 스레드를 직접 늘리면 시스템 전체가 느려지지만, Go의 M:N 모델을 응용하면 시스템 성능에 거의 영향을 주지 않으면서(Low Impact) 정밀한 감시를 수행할 수 있다.

[7] Go Concurrency: 채널과 통신(Channel & CSP 모델)

[01] Unbuffered Channel: 동기적 데이터 교환과 고루틴 블로킹 메커니즘 분석

[01-1] 100 개의 고루틴이 들어갈 때, 리눅스 커널이 관리하는 스레드는 몇 개인가?

- 랑데부(Rendezvous) 현상: 송신자와 수신자가 채널에서 만날 때까지 서로 기다린다는 것은 어떤 의미인가?
- 블로킹과 데드락: 받을 준비가 된 고루틴이 없는데 데이터를 보내려고 하면, 해당 고루틴은 어떤 상태가 되는가?

[01-2] 송신자와 수신자의 속도 차이를 의도적으로 만들어, 채널이 어떻게 두 고루틴의 발거음을 맞추는지 관찰

03_unbuffered_channel.go

```
package main

import (
    "fmt"
    "time"
)

func main() {
    // 1. 버퍼가 없는 채널 생성 (Unbuffered Channel)
    ch := make(chan string)

    // 2. 수신자 고루틴 (느린 일꾼)
    go func() {
        fmt.Println("[수신자] 3초 뒤에 데이터를 받을 준비가 됩니다...")
        time.Sleep(3 * time.Second)

        msg := <-ch // 데이터 수신 (송신자가 올 때까지 여기서 대기)
        fmt.Printf("[수신자] 데이터 수신 성공: %s\n", msg)
    }()

    // 3. 송신자 (메인 고루틴)
    fmt.Println("[송신자] 데이터를 채널에 던집니다...")

    // 버퍼가 없으므로 수신자가 <-ch 를 호출할 때까지 여기서 멈춤 (Blocking)
    ch <- "비밀 메시지"

    fmt.Println("[송신자] 수신자가 받아갔으므로 이제 다음 일을 합니다.")
}
```

```
// 결과 확인을 위해 잠시 대기
time.Sleep(1 * time.Second)
}
```

결과:

표준 출력 결과는 다음과 같다.

```
[송신자] 데이터를 채널에 던진다...
[수신자] 3 초 뒤에 데이터를 받을 준비가 된다...
[수신자] 데이터 수신 성공:비밀 메시지
[송신자] 수신자가 받아갔으므로 이제 다음 일을 한다.
```

결과 해석:

프로그램 실행 결과를 보면, [송신자] 데이터를 채널에 던진다...가 먼저 출력되지만 그 직후 다음 문장은 즉시 출력되지 않는다. 메인 고루틴은 `ch <- "비밀 메시지"` 코드에서 약 3 초 동안 멈춰 있게 된다. 이는 생성한 채널이 버퍼가 없는 채널이기 때문이다.

버퍼가 없는 채널은 데이터를 임시 저장하지 않는다. 따라서 송신자는 반드시 누군가 데이터를 받을 준비가 되어 있어야만 전송을 완료할 수 있다.

즉, 송신(`ch <- data`)과 수신(`<-ch`)이 동시에 만나야 통신이 성립한다.

이 특성 때문에 채널은 단순한 데이터 전달 도구가 아니라 고루틴 간의 동기화 장치 역할까지 수행한다.

학습에서는 수신자 고루틴이 먼저 실행되지만 `time.Sleep(3 * time.Second)` 때문에 일부러 3 초 동안 대기한다. 그동안 메인 고루틴은 채널 송신 시점에서 Block 상태로 멈춘다. 이후 수신자 고루틴이 `msg := <- ch`를 호출하는 순간, 송신자와 수신자가 서로 만나 데이터 전달이 즉시 이루어지고, 그제야 송신자의 블로킹이 해제되어 [송신자] 수신자가 받아갔으므로 이제 다음 일을 합니다. 가 출력된다.

즉, 이 학습은 “채널은 데이터가 실제로 소비될 때까지 송신자를 멈춘다”는 사실을 보여준다. 이는 공유 메모리 기반 동기화에서 흔히 발생하는 Race Condition 문제를 줄이고, 고루틴 간 실행 순서를 안전하게 맞추는 Go의 핵심 철학을 잘 보여주는 예제이다.

또한 수신자 고루틴을 제거한 상태에서 실행하면, 송신자는 영원히 데이터를 받을 상대를 기다리게 된다. Go 런타임은 모든 고루틴이 Block 상태에 빠졌다고 판단하면 `fatal error: all goroutines are asleep - deadlock!` 에러를 출력하며 프로그램을 강제 종료한다. 이는 Go 런타임이 채널 기반 동기화 오류를 스스로 탐지하는 안전장치를 가지고 있음을 의미한다.

[01-3] 이론

(1) Channel 기반 동기화 모델

Go의 채널(Channel)은 단순한 큐(Queue)가 아니라 고루틴 간 안전한 통신과 동기화를 동시에 제공하는 런타임 메커니즘이다.

채널을 통해 데이터를 전달할 때, 개발자는 직접 Mutex나 Lock을 다루지 않아도 된다.

Go는 “공유 메모리를 통해 통신하지 말고, 통신을 통해 메모리를 공유하라(Do not communicate by sharing memory; instead, share memory by communication)”라는 철학을 중심으로 설계되었다.

버퍼가 없는 채널은 송신과 수신이 반드시 동시에 만나야 한다.

송신자는 수신자가 등장할 때까지 Block 상태로 대기하며, 수신자 역시 송신자가 데이터를 보낼 때까지 기다린다. 이 방식은 고루틴 간 실행 순서를 자연스럽게 맞춰주므로 강력한 동기화 도구로 사용된다.

반면 버퍼가 있는 채널은 일정 개수만큼 데이터를 내부 큐에 저장할 수 있다.

버퍼가 가득 차기 전까지는 Block 되지 않으며, 수신자는 버퍼에 데이터가 존재하면 즉시 읽을 수 있다.

따라서 버퍼 채널은 Producer-Consumer 구조에서 처리량을 높이는데 유리하다.

(2) Blocking과 Deadlock

채널 송수신은 기본적으로 Blocking 연산이다.

- | |
|---|
| <ul style="list-style-type: none">- 송신 시: 데이터를 받을 고루틴이 없으면 송신자는 대기- 수신 시: 데이터가 들어올 때까지 수신자는 대기 |
|---|

이러한 특성은 안전한 동기화를 가능하게 하지만, 설계를 잘못하면 Deadlock(교착 상태)이 발생할 수 있다.

Deadlock은 모든 고루틴이 서로를 기다리며 영원히 진행되지 못하는 상태를 의미한다.

Go 런타임은 스케줄러 수준에서 이를 감지할 수 있으며, 실행 가능한 고루틴이 하나도 없으면 프로그램을 강제 종료한다.

이는 C 언어의 pthread 환경과 차별되는 중요한 특징이다. C에서는 데드락이 발생해도 운영체제가 자동으로 감지하지 못하는 경우가 많아 프로그램이 단순히 “멈춘 것처럼” 보인다. 하지만 Go는 런타임 차원에서 이를 감지하고 즉시 오류를 출력해 개발자가 문제를 빠르게 파악할 수 있도록 돕는다.

(3) CSP(Communication Sequential Process) 모델

Go 채널은 CSP 이론의 영향을 강하게 받았다. CSP는 프로세스들이 공유 메모리를 직접 건드리지 않고, 메시지 전달(Message Passing)을 통해 협력해야 한다는 병렬 프로그래밍 모델이다.

기존 멀티스레드 환경에서는 여러 스레드가 동일한 메모리를 동시에 수정하면 Race Condition, Deadlock, Priority Inversion 같은 복잡한 문제가 빈번하게 발생했다. Go는 채널을 통해 데이터 흐름 자체를 통제함으로써 이러한 문제를 줄이고자 했다.

결국 채널은 단순한 데이터 전달 통로가 아니라

- 실행 흐름 제어
- 작업 순서 보장
- 안전한 동기화
- 병렬 작업 조율

까지 담당하는 Go 동시성 모델의 핵심 구성 요소라고 볼 수 있다.

(4) 채널(Channel) vs C 언어의 파이프(Pipe)

C 언어의 파이프(Pipe)는 운영체제 커널이 제공하는 IPC 메커니즘이다.

프로세스 간 데이터를 전달하기 위해 커널 내부의 파이프 버퍼를 사용하며, read()와 write() 시스템 콜을 통해 데이터를 주고 받는다.

반면, Go의 채널(Channel)은 운영체제 커널이 아니라, Go 런타임 내부 자료구조로 구현된다. 즉, 대부분의 채널 통신은 유저 레벨에서 처리되며, 커널 모드 전환이 최소화된다.

이 차이 때문에 채널은 일반적인 Pipe보다 훨씬 가볍고 빠르게 동작할 수 있다. 특히 같은 프로세스 내부 고루틴 간 통신에서는 커널 개입이 거의 없으므로 매우 낮은 비용으로 동기화와 데이터 교환을 수행할 수 있다.

또한, 채널은 단순한 데이터 통로 이상의 의미를 가진다.

C에서 멀티스레드 프로그래밍을 할 때는 공유 메모리를 보호하기 위해 Mutex를 사용해야 하고, Lock 순서를 잘못 설계하면 Deadlock이나 Race Condition 문제가 발생할 수 있다.

하지만 Go 채널은 데이터 자체를 전달하는 방식으로 동작하기 때문에 “누가 이 데이터를 소유하고 있는가?”가 명확해진다. 데이터를 보낸 쪽은 더 이상 해당 데이터를 직접 건드리지 않고, 받은 고루틴이 새로운 소유권을 갖게 된다.

이를 Ownership Transfer라고 한다.

[01-4] 추가

(1) 동기화 버그 방지

보안 모듈 개발 시 멀티스레드 환경에서 가장 무서운 것이 Race Condition이다.

뮤텍스를 깜빡하고 안 걸면 데이터가 깨지지만, Go에서는 채널을 통해 데이터를 전달하는 구조를 잡으면 구조적으로 데이터 경쟁을 방지할 수 있다.

실시간 파일 스캔 로그를 수집 서버로 보낼 때, 수집 서버가 바빠서 못받고 있다면 송신 고루틴을 자연스럽게 블로킹시켜 데이터 유실을 방지하거나, 채널의 상태를 모니터링하여 시스템 병목 지점을 빠르게 파악할 수 있다.

[02] Unbuffered Channel: 동기적 데이터 교환과 고루틴 블로킹 메커니즘 분석

[02-1] 100 개의 고루틴이 들어갈 때, 리눅스 커널이 관리하는 스레드는 몇 개인가?

- 랑데부(Rendezvous) 현상: 송신자와 수신자가 채널에서 만날 때까지 서로 기다린다는 것은 어떤 의미인가?
- 블로킹과 데드락: 받을 준비가 된 고루틴이 없는데 데이터를 보내려고 하면, 해당 고루틴은 어떤 상태가 되는가?

[02-2] 송신자와 수신자의 속도 차이를 의도적으로 만들어, 채널이 어떻게 두 고루틴의 발거음을 맞추는지 관찰

04_buffered_select.go

```
package main
import (
    "fmt"
    "time"
)

func main(){
    // 1. 버퍼가 3개인 채널 생성
    // 수신자가 없어도 3개까지 송신자 멈추지 않음
    logChan := make(chan string, 3)
    quitChan := make(chan bool)

    // 2. 로그 처리기(수신자)
    go func(){
        for {
            select {
            case log := <- logChan:
                fmt.Printf("[로그 서버] 처리 중: %s\n", log)
                time.Sleep(1 * time.Second)

            case <- quitChan:
                fmt.Println("[로그 서버] 종료 요청 수신. 시스템을 닫습니다.")
                return

            case <- time.After(5 * time.Second):
                fmt.Println("[경고] 5초간 활동이 없다. 대기 모드 진입")
            }
        }
    }()
}
```

```
// 3. 로그 생성기(송신자)
fmt.Println("[생성기] 로그 3개를 빠르게 던진다.")
logChan <- "로그 #1"
logChan <- "로그 #2"
logChan <- "로그 #3"
fmt.Println("[생성기] 버퍼가 있어서 멈추지 않고 즉시 다음 작업을 진행")

time.Sleep(5 * time.Second)
quitChan <- true
time.Sleep(1 * time.Second)
}
```

결과:

표준 출력은 다음과 같다.

```
[생성기] 로그 3개를 빠르게 던진다.
[생성기] 버퍼가 있어서 멈추지 않고 즉시 다음 작업을 진행
[로그 서버] 처리 중: 로그 #1
[로그 서버] 처리 중: 로그 #2
[로그 서버] 처리 중: 로그 #3
[로그 서버] 종료 요청 수신. 시스템을 닫습니다.
```

결과 해석:

실행 결과를 보면 [생성기] 로그 3개를 빠르게 던진다. 출력 직후, 로그 서버가 아직 하나도 처리하지 않았음에도 [생성기] 버퍼가 있어서 멈추지 않고 즉시 다음 작업을 한다. 가 바로 출력되는 것을 확인할 수 있다.

이는 `logChan := make(chan string, 3)`으로 생성한 Buffered Channel 이 내부적으로 3 개의 데이터를 저장할 수 있는 버퍼 공간을 가지고 있기 때문이다.

Unbuffered Channel에서는 송신과 수신이 동시에 만나야 했지만, Buffered Channel에서는 버퍼가 비어 있는 동안 송신자는 데이터를 큐에 밀어 넣고 즉시 다음 작업으로 진행할 수 있다.

즉, 송신자와 수신자의 실행 속도가 서로 달라도 시스템이 멈추지 않고 비동기적으로 동작할 수 있다.

이후 로그 서버 고루틴은 `select` 문을 통해 `logChan`에 들어온 데이터를 하나씩 읽어 처리한다. 각 로그 처리마다 `time.Sleep(1 * time.Second)`가 있으므로 처리 속도는 느리지만, 이미 버퍼 안에 데이터가 저장되어 있으므로 송신자는 영향을 받지 않는다.

결과적으로 생산자는 빠르게 로그를 생성하고, 소비자는 자신의 속도에 맞춰 천천히 처리하는 Producer-Consumer 구조가 자연스럽게 형성된다.

또한, select 문은 여러 채널을 동시에 감시하는 역할을 수행한다.

logChan 에 데이터가 들어오면 로그를 처리하고, quitChan 에 종료 신호가 들어오면 시스템을 종료하며, 둘 다 일정 시간 동안 아무 이벤트가 없으면 time.After 가 활성화되어 타임아웃 경고를 출력한다. 이는 하나의 스레드가 여러 이벤트를 동시에 감시하는 이벤트 기반(Event-driven)구조와 유사하다.

만약 이 학습에서 로그를 4 개 이상 연속으로 넣었다면 상황이 달라진다.

버퍼 크기는 3 이므로 네 번째 송신 시점에는 버퍼가 가득 찬 상태가 된다.

이 경우 송신자는 소비자가 데이터를 하나 꺼내갈 때까지 Block 상태로 대기하게 된다. 즉, Buffered Channel 은 단순히 성능 향상을 위한 큐가 아니라, 시스템 처리량을 넘어서는 데이터 폭주를 자동으로 제어하는 Back-pressure(역압) 메커니즘 역할까지 수행한다.

[02-3] 이론

(1) Buffered Channel 과 Producer-Consumer 패턴

Buffered Channel 은 내부 버퍼를 가진 채널이다. 버퍼 크기만큼 데이터를 임시 저장할 수 있기 때문에 송신자와 수신자가 정확히 동시에 동작하지 않아도 된다.

이 구조는 대표적인 Producer-Consumer 패턴 구현에 매우 적합하다.

- | |
|--|
| <ul style="list-style-type: none">- Producer(생산자): 데이터를 생성하여 채널에 넣는 역할- Consumer(소비자): 채널에서 데이터를 꺼내 처리하는 역할 |
|--|

Unbuffered Channel 에서는 생산자와 소비자가 서로를 직접 기다려야 했지만, Buffered Channel 에서는 버퍼가 중간 저장소 역할을 수행한다. 따라서 생산자는 소비자의 처리 속도와 관계없이 일정량까지는 계속 작업을 수행할 수 있다.

이러한 비동기 구조는 시스템 전체 응답성을 크게 향상시킨다. 예를 들어, 로그 수집 시스템, 네트워크 패킷 처리기, 보안 이벤트 큐같은 환경에서는 데이터 생산 속도가 순간적으로 폭증할 수 있는데, 버퍼는 이를 흡수하여 시스템이 즉시 멈추지 않도록 도와준다.

하지만 버퍼가 무한정 데이터를 저장할 수 있는 것은 아니다. 버퍼가 가득 차면 송신자는 자동으로 Block 상태에 들어간다. 이를 Back-pressure(역압)라고 한다.

Back-pressure 는 매우 중요한 안정성 메커니즘이다. 만약 송신자를 멈추지 않는다면, 데이터가 끝없이 쌓이며 메모리가 폭발적으로 증가하고 결국 시스템 전체가 마비될 수 있다. Buffered Channel 은 처리 가능한 속도를 넘어서는 데이터 폭주를 자동으로 제어하여 시스템 안정성을 유지한다.

(2) select 문과 입출력 다중화(I/O Multiplexing)

Go 의 select 문은 여러 채널을 동시에 감시할 수 있는 동시성 제어 구조이다.

여러 송수신 연산 중 “가장 먼저 준비된 작업”하나를 선택하여 실행한다.

이는 리눅스의

- select()
- poll()
- epoll()

같은 I/O Multiplexing 시스템 콜과 개념적으로 매우 유사하다.

C 언어에서 여러 소켓을 동시에 감시하려면 개발자가 직접 FD 집합을 관리하고, 이벤트 루프를 설계하며, epoll 구조체와 이벤트 테이블을 조작해야 한다.

하지만 Go 는 이러한 복잡한 과정을 런타임 내부로 숨겨버렸다.

Go 런타임은 내부적으로 OS 의

- Linux: epoll
- BSD/macOS: Kqueue
- Windows: IOCP

같은 고성능 이벤트 시스템을 사용한다. 이를 Network Poller 라고 부른다.

개발자는 단순히 select 문만 사용하면 되지만, 실제 내부에서는 Go 런타임이 수많은 고루틴과 네트워크 이벤트를 효율적으로 스케줄링한다.

즉, Go 의 select 는

- 여러 이벤트 감시
- 타임아웃 처리
- 채널 동기화
- 비동기 이벤트 루프

를 매우 간결한 문법 안에서 제공하는 고수준 동시성 도구라고 볼 수 있다.

[02-4] 추가

(1) 대규모 로그 수집 엔진

수만 대의 PC 에서 보안 로그가 쏟아질 때, 버퍼드 채널은 일시적 트래픽 폭주(Burst traffic)를 흡수하는 완충지대 역할을 한다.

(2) 실시간 대응 타임아웃

select 의 타임아웃 기능을 이용하면, 악성 코드 검사 모듈이 일정 시간 내에 응답을 주지 않을 때 시스템 전체가 멈추지 않도록 강제로 연결을 끊거나 예외 처리를 하는 장애 내구성을 쉽게 구현할 수 있다.

(3) 비정상 행위 탐지

여러 채널(네트워크, 파일, 레지스트리 감시 채널)을 select 로 묶어서 관찰하다가, 특정 조합의 이벤트가 발생할 때 즉시 대응하는 복합 이벤트 처리(CEP)로직에 최적이다.

[8] Go Concurrency: 데이터 경쟁(Race Condition) 자동 탐지

[01] Go Race Detector: 런타임 분석을 통한 데이터 경쟁 자동 탐지 및 분석

[01-1] C 언어에서 육안으로 찾기 힘들었던 데이터 오염을 Go 는 어떻게 도구 레벨에서 찾아내는가?

- 탐지 메커니즘: 코드가 실행되는 도중에 메모리 접근 패턴을 분석하여 데이터 경쟁을 찾아내는 Go 의 기술은 무엇인가
- 디버깅 효율성: 눈에 보이지 않는 비결정적인 버그 `-race` 플래그를 어떻게 시각화하여 보여주는가?

[01-2] 의도적으로 데이터 레이스가 발생하는 코드를 작성([3]pthread 편의 [02])

05_race_detector.go

```
package main

import (
    "fmt"
    "sync"
)

func main(){
    var count = 0
    var wg sync.WaitGroup

    // 2개의 고루틴이 하나의 변수 count를 마구 수정함(Lock 없음)
    for i := 0; i < 1000; i++){
        wg.Add(1)
        go func(){
            defer wg.Done()
            count++; // 데이터 경쟁 발생 지점
        }()
    }

    wg.Wait()
    fmt.Printf("최종 결과값: %d\n", count)
}
```

결과:

(1) 표준 출력 결과는 다음과 같다.

```
최종 결과값: 1000
최종 결과값: 999
최종 결과값: 962
```

(2) Race Detector 켜고 실행한 결과는 다음과 같다.

```
go run -race main.go
```

```
=====
WARNING: DATA RACE
Read at 0x00c000092028 by goroutine 9:
    main.main.func1()
        /home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:17 +0x7b

Previous write at 0x00c000092028 by goroutine 10:
    main.main.func1()
        /home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:17 +0x8d

Goroutine 9 (running) created at:
    main.main()
        /home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:15 +0x78

Goroutine 10 (finished) created at:
    main.main()
        /home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:15 +0x78
=====
WARNING: DATA RACE
Write at 0x00c000092028 by goroutine 771:
    main.main.func1()
        /home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:17 +0x8d

Previous write at 0x00c000092028 by goroutine 1006:
    main.main.func1()
        /home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:17 +0x8d

Goroutine 771 (running) created at:
    main.main()
        /home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:15 +0x78

Goroutine 1006 (running) created at:
    main.main()
        /home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:15 +0x78
=====
최종 결과값: 1000
Found 2 data race(s)
exit status 66
```

출력 결과를 보면 Go 의 Race Detector 가 단순히 “문제가 있다”라고만 알려주는 것이 아니라, 어떤 고루틴이 언제 같은 메모리에 접근했는지를 구체적으로 추적하고 있음을 확인할 수 있다.

첫 번째 경고에서는 다음과 같은 상황이 발생했다.

```
WARNING: DATA RACE
```

```
Read at 0x00c000092028 by goroutine 9:
```

고루틴이 9 번이 특정 메모리 주소(0x00c000092028)의 값을 읽었다는 의미이다.

그런데 바로 아래에서

```
Previous write at 0x00c000092028 by goroutine 10:
```

고루틴 10 번이 동일한 메모리 주소에 값을 쓰고 있었다는 사실이 발견된다.

즉, 두 개의 고루틴이 동시에 count 변수에 접근하고 있었으며, 하나는 읽고(Read), 다른 하나는 쓰기(Write)를 수행하고 있었다.

이 순간 어떤 고루틴이 먼저 실행될지 보장되지 않으므로 데이터 경쟁(Data Race)이 발생한다.

특히 Race Detector 는 단순히 “경쟁이 발생했다”에서 끝나지 않고 다음 정보까지 모두 제공한다.

- 어떤 고루틴이 접근했는지
- 읽기(Read)인지 쓰기(Write)인지
- 어떤 메모리 주소에서 충돌했는지
- 소스 코드 몇 번째 줄에서 발생했는지
- 해당 고루틴이 어디서 생성되었는지

예를 들어,

```
/home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:17
```

이는 실제 문제 지점이 count++가 있는 17 번째 줄이라는 뜻이다.

또한 아래 메시지는:

```
Goroutine 9 (running) created at:
```

```
main.main()
```

```
/home/pumpkinbee/os_lab/chapter_6/5_race_detector/main.go:15
```

해당 고루틴이 main.go 의 15 번째 줄에서 생성되었다는 의미이다.

즉, Go 런타임은 “문제가 발생한 위치”뿐 아니라 “누가 그 고루틴을 만들었는지”까지 역추적해 보여준다.

두 번째 경고에서는 더욱 심각한 상황이 발생한다.

```
Previous write at 0x00c000092028 by goroutine 1006:
```

이번에는 읽기/쓰기가 아니라 두 개의 고루틴이 동시에 같은 변수에 쓰기(Write)를 수행하고 있다.

count++는 실제로 CPU 수준에서는 다음과 같은 세 단계로 나뉜다.

1. 메모리에서 count 읽기
2. 값 + 1 계산
3. 다시 메모리에 저장

만약 두 고루틴이 동시에 실행되면:

- 둘 다 현재 값 5를 읽고
- 각각 6으로 계산한 뒤
- 둘 다 6을 저장

하는 상황이 발생할 수 있다.

결과적으로 증가 연산이 한 번 사라지게 되면, 이것이 최종 결과가 매번 달라지는 이유다.
실제로 실행 결과를 보면

최종 결과값: 1000
최종 결과값: 999
최종 결과값: 962

처럼 실행할 때마다 값이 달라진다.

이는 프로그램의 논리 자체가 깨졌다는 의미이며, 멀티스레드/고루틴 환경에서 동기화 없이 공유 데이터를 수정하는 것이 얼마나 위험한지를 보여준다.

마지막 줄

Found 2 data race(s)
exit status 66

는 Go 런타임이 총 2개의 데이터 경쟁 상황을 발견했음을 의미하며, 프로그램을 비정상 종료 상태(exit status 66)로 종료시킨다.

즉, Go의 Race Detector는 실행 중 메모리 접근을 추적하여 경쟁 상태를 실시간으로 탐지하는 동적 분석 도구이며, 대규모 동시성 서버 개발에서 매우 강력한 디버깅 무기가 된다.

[01-3] 이론

(1) Go Race Detector의 동작 원리

Go의 Race Detector는 단순히 결과값이 이상한지를 보는 것이 아니라, 프로그램 실행 중 모든 메모리 접근 흐름을 추적하여 동시 접근 충돌 여부를 분석하는 동적 분석(Dynamic Analysis) 도구이다.

핵심 원리는 인스트루멘테이션(Instrumentation)이다.

go run -race 옵션으로 컴파일하면 Go 컴파일러는 원래 코드에 추가적인 감시 코드를 자동 삽입한다.

예를 들어, 단순한

```
count++
```

코드도 내부적으로 다음과 같은 감시 흐름이 추가된다.

- 어떤 고루틴이 접근했는지
- 읽기(Read)인지 쓰기(Write)인지
- 어떤 메모리 주소인지
- 접근 시점이 언제인지
- Mutex, Channel 등의 동기화 도구를 거쳤는지

이러한 정보가 런타임 동안 계속 기록된다.

프로그램이 실행되면 Go 런타임은 각 고루틴의 메모리 접근 기록을 실시간으로 비교한다.

이때 서로 다른 고루틴이 동일한 메모리 주소에 동시에 접근하면서, 최소 하나 이상이 Write 작업이고, 그 사이에 Mutex 나 Channel 같은 동기화 흔적이 존재하지 않으면 이를 Data Race 로 판단한다.

즉, Race Detector 는 단순한 “값 비교”가 아니라

- 메모리 접근 순서(Happens-before)
- 동기화 이벤트
- 고루틴 실행 흐름

까지 모두 추적하여 논리적으로 안전한 접근인지 검증하는 시스템이다.

또한, Race Detector 는 문제 발생 시 다음 정보까지 제공한다.

- 어떤 고루틴끼리 충돌했는지
- Read/Write 중 어떤 작업이었는지
- 실제 메모리 주소
- 문제 발생 코드 라인번호
- 해당 고루틴 생성 위치(Call Stack)

이 때문에 대규모 동시성 서버에서도 매우 강력한 디버깅 도구로 사용된다.

(2) Race Detector 의 트레이드 오프(Trade-off)

Race Detector 는 매우 강력하지만 비용이 크다.

감시 코드를 계속 삽입하고 모든 메모리 접근을 추적해야 하기 때문에

- 실행 속도가 약 2~10 배 정도 느려질 수 있고
- 메모리 사용량 역시 크게 증가한다.

즉, 프로그램의 모든 읽기/쓰기 동작마다 “기록”과 “검사”가 추가되므로 런타임 오버헤드가 매우 커진다.

따라서 실제 서비스 운영 환경에서는 거의 사용하지 않는다.

대신 다음 환경에서는 주로 사용된다.

- 개발 단계
- 테스트 자동화(CI/CD)
- 동시성 버그 검증
- 서버 안정성 테스트
- 대규모 리팩토링 검증

특히 Go 기반 서벌 개발에서는

```
go test -race
```

명령이 표준처럼 사용되며, 배포 전에 데이터 경쟁 문제를 조기에 발견하는 핵심도구로 활용된다.

[01-4] 추가

(1) 보안 취약점으로서의 데이터 레이스

데이터 레이스는 단순히 값이 틀리는 문제가 아니다. 보안 관점에서는 TOCTOU 공격의 빌미가 된다.

예를 들어, 보안 엔진이 “이 파일은 안전함”이라고 체크한 변수 값을 메모리에 저장했는데, 찰나의 순간에 다른 고루틴이 그 값을 “위험함”으로 바꿔버린다면 엔진의 판단이 꼬일 수 있다.

(2) 실무?

기업에서는 수천 줄의 동시성 코드가 포함된 보안 에이전트를 배포하기 전에, 자동화된 테스트 시스템(CI)에서 `-race` 플래그를 필수로 켜고, 이를 통해 배포 후 발생할 수 있는 치명적인 시스템 크래시나 보안 우회 가능성을 사전에 100% 차단하는 프로세스를 갖춘다.

[02] Mutex vs Channel: 어떤 상황에 어떤 동기화 도구를 쓸 것인가?

[02-1] 공유 메모리 방식(Mutex)과 메시지 패싱 방식(Channel)의 성능 및 코드 가독성 차이는?

- 소유권: 데이터를 '보호'하는 것(Mutex)과 데이터를 '전달'하는 것(Channel)의 구조적 차이는 무엇인가?
- 성능과 가독성: 단순한 카운팅 작업에서 어떤 도구가 더 빠르고, 복잡한 상태 관리에서는 어떤 도구가 더 유지보수하기 좋은가?

[02-2] 두 가지 방식으로 동일한 카운팅 작업을 수행하여 결과를 비교

06_mutex_vs_channel.go

```
package main

import (
    "fmt"
    "sync"
    "time"
)

// 1. Mutex 방식: 공유 메모리를 락으로 보호
func useMutex(iterations int){
    var mu sync.Mutex
    count := 0
    var wg sync.WaitGroup

    start := time.Now()
    for i:= 0; i < iterations; i++){
        wg.Add(1)
        go func(){
            defer wg.Done()
            mu.Lock()
            count++
            mu.Unlock()
        }()
    }

    wg.Wait()
    fmt.Printf("[Mutex] 결과: %d, 소요시간: %v\n", count, time.Since(start))
}

// 2. Channel 방식: 데이터를 채널로 전달하여 처리
func useChannel(iterations int){
    count := 0
    ch := make(chan int)
    var wg sync.WaitGroup

    start := time.Now()

    go func(){
        for range ch {
            count++
        }
    }()

    for i:= 0; i < iterations; i++){
        ch<>
```

```

        wg.Done()
    }
}()

for i := 0; i < iterations; i++{
    wg.Add(1)
    ch <- 1
}
wg.Wait()
close(ch)
fmt.Printf("[Channel] Result: %d, 소요시간: %v\n", count, time.Since(start))
}

func main(){
    const iterations = 100000
    fmt.Println("--- Mutex vs Channel 성능 비교 ---")
    useMutex(iterations)
    useChannel(iterations)
}

```

결과:

표준 출력 결과는 다음과 같다.

```

--- Mutex vs Channel 성능 비교 ---
[Mutex] 결과: 100000, 소요시간: 46.420708ms
[Channel] Result: 100000, 소요시간: 46.808981ms

```

결과 해석:

이번 학습에서는 동일한 작업(100,000 번 카운트 증가)을 수행할 때 Mutex 방식과 Channel 방식의 성능 및 설계 차이를 비교하였다.

실행 결과는 위와 같다.

두 방식 모두 최종 결과값은 정확하게 100000 이 출력되었다.

이는 두 방식 모두 데이터 경쟁없이 안전하게 공유 데이터를 처리했음을 의미한다.

성능 측면에서는 Mutex 방식이 Channel 방식보다 약간 더 빠르게 동작하였다.

Mutex 는 단순히

1. Lock 획득
2. count++
3. Unlock

만 수행하면 되므로 오버헤드가 비교적 적다.

반면, Channel 방식은 내부적으로

- 채널 큐 관리
- 송수신 동기화
- 고루틴 스케줄링
- wake-up 처리

등의 추가 작업이 필요하다.

즉, Channel은 단순 메모리 보호 장치라기보다 “메시지 전달 기반 동시성 시스템”에 가깝기 때문에 내부 구조가 더 복잡하다.

하지만 설계 관점에서는 큰 차이가 존재한다.

Mutex 방식에서는 공유 변수 count를 여러 고루틴이 직접 접근한다.

따라서 개발자는 코드 곳곳에서 Lock/Unlock을 정확하게 관리해야 하며, 실수 시 다음 문제가 발생할 수 있다.

- Deadlock
- Double Lock
- Unlock 누락
- Race Condition

즉, 데이터는 공유되고 있으며, Mutex는 그 공유 자원에 대한 “출입 통제 장치” 역할을 수행한다.

반면 Channel 방식에서는 count를 오직 하나의 Manager 고루틴만 수정한다.

다른 고루틴들은 데이터를 직접 건드리지 않고,

```
ch <- 1
```

처럼 “메시지”만 전달한다.

즉, 데이터 자체를 공유하지 않고 소유권을 특정 고루틴에게 집중시키는 구조이다.

이 방식은

- 데이터 흐름이 명확하고
- Race Condition 가능성이 줄어들며
- 동시성 설계가 직관적

이라는 장점이 있다.

결과적으로 이번 학습을 통해

- 단순한 공유 변수 보호에는 Mutex가 더 빠르고 효율적이며
- 복잡한 비동기 이벤트 처리나 Producer-Consumer 구조에서는 Channel 기반 설계가 더 안전하고 직관적이라는 점을 확인할 수 있었다.

[02-3] 이론

(1) 공유 메모리(Shared Memory) 기반 동기화 - Mutex 방식

Mutex 방식의 핵심 철학은 “데이터는 한 곳에 그대로 두고, 여러 작업자(Thread/Goroutine)가 번갈아 접근한다.”이다.

즉, 여러 고루틴이 동일한 메모리 공간을 직접 공유하며 사용한다. 이때 동시에 접근하면 데이터가 깨질 수 있으므로 Mutex 가 “한 번에 한 명만 들어오게 하는 잠금장치(Lock)”역할을 수행한다.

예를 들어,

```
mu.Lock()
count++
mu.Unlock()
```

코드는

- Lock 을 획득한 고루틴만 count 수정 가능
- 작업이 끝나면 Unlock 으로 다른 고루틴에게 권한 전달

이라는 구조로 동작한다.

이 방식의 가장 큰 장점은 매우 가볍고 빠르다는 점이다.

단순한 변수 증가나 짧은 임계구역(Critical Section)에서는 채널보다 훨씬 효율적이며, 실제 운영체제 커널이나 데이터베이스 엔진도 내부적으로 Mutex 를 광범위하게 사용한다.

하지만 단점도 존재한다.

공유 메모리 방식에서는 개발자가 직접 Lock/Unlock 흐름을 관리해야 하기 때문에

- Unlock 누락
- 중복 lock
- Lock 순서 충돌
- Race Condition
- Deadlock

같은 문제가 발생하기 쉽다.

즉, 성능은 좋지만 설계 복잡도가 증가하며, 실수에 매우 취약한 방식이다.

(2) 메시지 패싱(Message Passing) 기반 동기화 - Channel 방식

Go 의 Channel 방식은 CSP(Communication Sequential Processes) 모델 기반으로 설계되었다.

핵심 철학은 다음과 같다.

“데이터를 여러 작업자가 공유하지 말고, 데이터를 특정 작업자에게 전달하라”

즉, 데이터를 여러 고루틴이 동시에 건드리지 않고 데이터를 가진 고루틴만 해당 값을 수정한다.

예를 들어,

ch <- 1

은 공유 변수에 직접 접근하는 것이 아니라, “작업 요청” 자체를 메시지 형태로 전달하는 것이다.

이번 학습에서는 Manager 고루틴 하나만 count 를 수정하였기 때문에, 다른 고루틴들은 count 에 직접 접근하지 않는다.

즉,

- 데이터 소유권이 명확하고
- Race Condition 가능성이 줄어들며
- 흐름 자체가 이벤트 기반으로 구성된다.

이 방식의 가장 큰 장점은 동기화 자체가 자연스럽다는 점이다.

Mutex 처럼 Lock/Unlock 을 직접 관리하지 않아도

- 채널 송수신 자체가 동기화 역할을 수행하고
- 데이터 흐름이 코드 상에 드러나며
- 비동기 파이프라인 설계가 쉬워진다.

특히

- 로그 처리 시스템
- 이벤트 큐
- 작업 분배 시스템
- 비동기 네트워크 서버

같은 복잡한 워크플로우에서 매우 강력하다.

하지만 단점도 존재한다.

채널은 내부적으로

- 큐 관리
- 고루틴 대기열
- wake-up 스케줄링
- 런타임 동기화

등의 복잡한 작업을 수행한다.

따라서, 단순 카운팅 같은 매우 짧은 작업에서는 Mutex 보다 오버헤드가 커질 수 있다.

즉,

- 단순 공유 변수 보호 -> Mutex 가 효율적
 - 복잡한 이벤트 흐름과 비동기 처리 -> Channel 이 직관적
- 이라는 차이가 존재한다.

즉, Go 의 철학처럼 “메모리를 공유하며 통신하지 말고, 통신함으로써 메모리를 공유하라”라는 문장이 바로 Channel 기반 설계를 가장 잘 설명하는 핵심 철학이다.

[02-4] 추가

(1) Thread-Safe 보안 설계

보안 솔루션 내부에서 ‘정책’ 데이터를 업데이트할 때, 뮤텝스를 쓰면 모든 스레드가 락이 풀리길 기다리며 병목 현상이 생길 수 있다.

이때 Channel 을 이용해 “정책 업데이트”라는 메시지를 뿌려주면(Broadcast), 각 스레드가 자신의 타이밍에 맞춰 안전하게 데이터를 갱신하는 이벤트 기반 설계가 가능해진다.

(2) 실무 조언

면접에서 ‘단순히 공유 자원의 값을 바꾸는 성능 중심 작업에는 Mutex 를 쓰겠지만, 복잡한 상태를 가진 프로세스 감시 로그나 네트워크 패킷의 흐름을 제어할 때는 소유권 전송이 명확한 Channel 을 사용하여 설계의 안정성을 높이겠습니다.’

[9] Go Concurrency: 고성능 네트워크

[01] net 패키지 해부: C 소켓 프로그래밍과의 추상화 레벨 비교

[01-1] C 언어에서 육안으로 찾기 힘들었던 데이터 오염을 Go 는 어떻게 도구 레벨에서 찾아내는가?

- 추상화의 위력: C 에서 수십 줄이었던 socket-bind-listen-accept 과정이 Go 에서는 어떻게 단 한줄로 요약되는가?
- 고성능 비밀: Go 는 어떻게 “클라이언트당 고루틴 하나”라는 단순한 모델로 수십만 개의 동시 접속을 효율적으로 처리하는가?(리눅스 epoll 과의 연결 고리)

[01-2] 가장 기초적인 TCP 에코 서버를 Go 로 작성

07_go_tcp_server.go

```
package main
import (
    "fmt"
    "bufio"
    "net"
)

func handleConnection(conn net.Conn){
    defer conn.Close()
    fmt.Printf("[서버] 클라이언트 접속: %s\n", conn.RemoteAddr().String())

    scanner := bufio.NewScanner(conn)
    for scanner.Scan() {
        msg := scanner.Text()
        fmt.Printf("[수신] %s\n", msg)

        // 에코(받은 메시지 그대로 돌려주기)
        conn.Write([]byte("Echo: " + msg + "\n"))
    }

    if err := scanner.Err(); err != nil {
        fmt.Println("읽기 에러:", err)
    }
    fmt.Println("클라이언트 연결 종료")
}

func main(){
```

```
// 1. listen(socket + bind + listen을 한번에 처리)
ln, err := net.Listen("tcp", ":9999")

if err != nil{
    fmt.Println("에러:", err)
    return
}

defer ln.Close()
fmt.Println("[서버] Go 에코 서버 시작(Port9999)...")
for {
    // 2. Accept(새로운 접속 대기)
    conn, err := ln.Accept()

    if err != nil{
        continue
    }

    // 3. 고루틴 생성(접속마다 독립된 일꾼 배정)
    // c언어의 fork()나 pthread_create()보다 압도적으로 가볍고 빠름
    go handleConnection(conn)
}
}
```

결과:

표준 출력은 다음과 같다.

```
[서버] Go 에코 서버 시작(Port9999)...
[서버] 클라이언트 접속: 127.0.0.1:47648
[수신] hi
[수신] hello
```

strace 실행 명령어:

```
strace -f go run main.go
```

결과 해석:

서버 실행 후 여러 클라이언트가 동시에 접속하자, 서버는 각 연결마다 새로운 고루틴을 생성하여 독립적으로 통신을 처리하였다.

클라이언트에서 보낸 “hi”, “hello” 메시지가 즉시 서버에 출력되었고, 동시에 각 클라이언트는 Echo 응답을 정상적으로 돌려받았다. 이를 통해 Go 의 net.Listener 와 고루틴 기반 동시성 모델이 정상적으로 동작함을 확인할 수 있었다.

또한 C 언어에서 직접 처리해야 했던 `socket()`, `bind()`, `listen()`, `accept()` 과정이 Go 의 `net.Listen()` 하나로 추상화되어 코드가 매우 단순해졌다.
개발자는 복잡한 소켓 주소 구조체(`sockaddr_in`) 초기화나 `htons()` 같은 네트워크 바이트 변환을 직접 처리하지 않아도 된다.

특히 접속마다 `go handlerConnection(conn)` 형태로 고루틴을 생성했음에도 시스템 자원 사용량이 급격히 증가하지 않았다. 이는 Go 런타임이 M:N 스케줄링 기반으로 수많은 고루틴을 소수의 OS 스레드 위에서 효율적으로 실행하기 때문이다.

만약 동일한 구조를 C 언어의 `fork()` 기반 멀티프로세스 서버로 구현했다면, 프로세스 생성 비용과 메모리 사용량이 훨씬 커졌을 것이다.

[01-3] 이론

(1) Network Poller 와 Go 의 비동기 네트워크 처리 구조

C 언어에서 수천~수만 개의 네트워크 연결을 효율적으로 처리하려면 개발자가 직접 `select`, `poll`, `epoll` 같은 입출력 다중화(I/O Multiplexing) 시스템 콜을 다뤄야 한다.

특히 `epoll` 기반 이벤트 루프 서버는 성능은 뛰어나지만 코드 구조가 매우 복잡해지고 상태 관리(State Machine)까지 직접 구현해야 하는 어려움이 있다.

Go 는 이 문제를 런타임 내부의 Network Poller 가 자동으로 해결한다.

개발자는 단순히 동기 방식처럼 `conn.Read()` 또는 `conn.Write()`를 호출하면 된다.

겉보기에는 Blocking I/O 처럼 보이지만, 실제 내부에서는 Go 런타임이 소켓을 Non-Blocking 모드로 관리하고 있다.

예를 들어, 어떤 고루틴이 `conn.Read()`에서 데이터를 기다리게 되면, Go 런타임은 해당 고루틴만 잠시 잠재운다(Park).

중요한 점은 이때 리눅스 커널 스레드(M)는 멈추지 않는다는 것이다.

커널 스레드는 즉시 다른 실행 가능한 고루틴을 처리하러 이동한다.

이후 네트워크 데이터가 실제로 도착하면, 리눅스 커널의 `epoll` 이 이벤트를 감지한다.

그러면 Go 의 Network Poller 가 “이 소켓에 데이터가 들어왔다”는 사실을 확인하고, 잠들어 있던 고루틴을 다시 실행 큐(Run Queue)에 넣는다.

결국 개발자는 매우 단순한 동기 코드만 작성했지만, 내부적으로는 고성능 비동기 이벤트 기반 서버가 동작하게 되는 것이다.

즉, Go 는 다음 두 가지를 동시에 달성한다.

- | |
|---|
| <ul style="list-style-type: none">- 개발자 관점: 읽기 쉽고 직관적인 동기 코드- 런타임 내부: <code>epoll</code> 기반의 고성능 비동기 네트워크 처리 |
|---|

이 구조 덕분에 Go 는 적은 코드만으로도 대규모 동시 접속 서버를 효율적으로 구현할 수 있다.

(2) One Goroutine per Connection 모델

전통적인 C 언어 서버에서는 “클라이언트 하나당 스레드 하나”모델이 매우 위험했다.

리눅스의 일반적인 pthread 는 생성 시 MB 크기의 고정 스택 메모리를 할당받기 때문에, 수천개만 생성해도 메모리 사용량이 폭발적으로 증가한다.

또한, 스레드 수가 많아질수록 커널의 Context Switching 비용도 커져 시스템 전체 성능이 급격히 저하된다.

반면, Go 의 고루틴 생성 시 약 2KB 수준의 매우 작은 스택으로 시작한다.

또한, Go 런타임이 필요에 따라 스택 크기를 동적으로 확장(Growable Stack)하기 때문에, 사용하지 않는 메모리를 미리 낭비하지 않는다.

그 결과, Go 에서는 클라이언트 연결마다 고루틴 하나를 할당하는 “One Goroutine per Connection” 모델이 오히려 가장 자연스럽고 효율적인 표준 구조가 된다.

```
go handleConnection(conn)
```

위 코드 한줄만으로 각 클라이언트가 독립적으로 처리된다.

어떤 클라이언트가 네트워크 지연으로 멈춰 있더라도, 다른 고루틴들을 영향을 받지 않고 계속 실행한다.

이 구조는 다음과 같은 장점을 가진다.

- 코드가 직관적이고 유지보수가 쉬움
- 연결마다 독립된 실행 흐름 보장
- 높은 동시 접속 처리 성능
- 적은 메모리 사용량
- epoll 기반 비동기 처리 자동 활용

실제로 Go 는 이러한 구조 덕분에 웹 서버, API Gateway, 실시간 채팅 서버, 클라우드 인프라, 보안 에이전트 등 수많은 네트워크 중심 시스템에서 널리 사용된다.

[01-4] 추가

(1) 강력한 기본 보안

Go 의 net 패키지는 메모리 안전성이 보장되므로, C 언어 소켓 프로그래밍에서 흔히 발생하는 Buffer Overflow 공격으로부터 원천적으로 안전하다.

(2) 에이전트 통신

백신 에이전트가 중앙 서버와 통신할 때, 네트워크 상태가 불안정하거나 수만 대가 동시에 접속하는 상황에서도 Go 의 내장된 타임아웃 처리와 효율적인 고루틴 관리 서비스 안정성을 비약적으로 높여준다.

(3) 분석 포인트

보안 엔지니어는 netstat 으로 소켓 상태를 볼 때, 수많은 연결이 있음에도 실제 프로세스의 스레드 개수가 적게 유지되는 이유를 Network Poller 이론으로 설명할 수 있어야 한다.

[02] net 패키지 해부: C 소켓 프로그래밍과의 추상화 레벨 비교

[02-1] C 언어에서 육안으로 찾기 힘들었던 데이터 오염을 Go 는 어떻게 도구 레벨에서 찾아내는가?

- 추상화의 위력: C 에서 수십 줄이었던 socket-bind-listen-accept 과정이 Go 에서는 어떻게 단 한줄로 요약되는가?
- 고성능 비밀: Go 는 어떻게 “클라이언트당 고루틴 하나”라는 단순한 모델로 수십만 개의 동시 접속을 효율적으로 처리하는가?(리눅스 epoll 과의 연결 고리)

[02-2] 가장 기초적인 TCP 에코 서버를 Go 로 작성

07_go_tcp_server.go

```
package main

import (
    "fmt"
    "net"
    "bufio"
)

// 클라이언트 정보를 담는 구조체
type client chan<- string // 메시지를 보낼 수 있는 전용 채널

var (
    entering = make(chan client)
    leaving = make(chan client)
    messages = make(chan string) // 모든 클라이언트가 보내는 메시지 공용 통로
)

// 1. Broadcaster: 모든 채팅 상태를 관리하는 '단일 소유자' 고루틴
func broadcast(){
    clients := make(map[client]bool) // 모든 접속자 명단(오직 이 고루틴만 접근함)
    for {
        select {
            case msg := <- messages:
                // 모든 접속자에게 메시지 전송
                for cli := range clients{
                    cli <- msg
                }
            case cli := <-entering:
                clients[cli] = true
            case cli := <-leaving:
                delete(clients, cli)
                close(cli)
        }
    }
}

// 2. HandleConn: 각 클라이언트와 대화하는 개별 일꾼 고루틴
func handleConn(conn net.Conn){
    ch := make(chan string)
```



```

    go clientWriter(conn, ch)

    who := conn.RemoteAddr().String()
    ch <- "당신의 주소는 " + who
    messages <- who + " 님이 입장하셨습니다."
    entering <- ch

    input := bufio.NewScanner(conn)
    for input.Scan(){
        messages <- who + ":" + input.Text()
    }

    leaving <- ch
    messages <- who + " 님이 퇴장하셨습니다."
    conn.Close()
}

func clientWriter(conn net.Conn, ch <- chan string){
    for msg := range ch {
        fmt.Println(conn, msg)
    }
}

func main(){
    fmt.Println("[서버] 고성능 채팅 서버 시작 (Port 9999)....")
    lisenter, _ := net.Listen("tcp", "localhost:9999")

    go broadcast() // 관리자 고루틴 실행

    for {
        conn, _ := lisenter.Accept()
        go handleConn(conn) // 접속마다 일꾼 고루틴 실행
    }
}

```

결과:

표준 출력 결과는 다음과 같다.

```

[서버] 고성능 채팅 서버 시작 (Port 9999)....
&{{0xc0000b2100}} 당신의 주소는 127.0.0.1:35912
&{{0xc0000b2100}} 127.0.0.1:35912:hi
&{{0xc0000b2100}} 127.0.0.1:35912:hello
&{{0xc0000b2180}} 당신의 주소는 127.0.0.1:34866
&{{0xc0000b2100}} 127.0.0.1:34866 님이 입장하셨습니다.
&{{0xc0000b2180}} 127.0.0.1:34866:hi
&{{0xc0000b2100}} 127.0.0.1:34866:hi
&{{0xc0000b2180}} 127.0.0.1:34866:hi
&{{0xc0000b2100}} 127.0.0.1:34866:hi
&{{0xc0000b2100}} 127.0.0.1:34866 님이 퇴장하셨습니다.

```

결과 해석:

서버 실행 후 여러 클라이언트가 동시에 접속하자, 각 사용자의 입장과 메시지가 모든 접속자에게 정상적으로 브로드캐스트되는 것을 확인할 수 있었다.

예를 들어, 127.0.0.1:34866 님이 입장하셨습니다.

[10] Go 네트워크 프로그래밍 및 고루틴 동기화

Go 언어의 `net.Listen` 을 통한 네트워크 서버 구현, 고루틴 기반의 동기화 기법 그리고 Go 런타임 내부의 GMP 스케줄링 작동 원리를 실무 및 아키텍처 관점에서 정리한다.

(1) 고루틴 동기화 기법: 채널과 뮤텍스의 올바른 활용

Go 언어는 “메모리를 공유해서 통신하지 말고, 통신을 해서 메모리를 공유하라”는 철학을 바탕으로 채널을 제공한다. 채널이 강력한 동기화 도구인 것은 사실이지만, 모든 상황에서 채널만 고집하는 것은 비효율적일 수 있다.

상황에 맞는 적절한 선택이 필요하다.

A. 채널 및 버퍼 활용(Channel)

- 주요 용도: 고루틴 간의 구조적인 흐름 제어, 데이터(작업 지시, 결과물) 전달, 작업 처리 속도 제한(Rate Limiting)
- 작동 특징: 버퍼가 없는 채널은 송수신 고루틴이 동시에 준비되어야 하는 동기적 랑데부 방식을 취한다. 반면 버퍼가 있는 채널은 지정된 크기만큼 데이터를 임시 보관할 수 있어 비동기적 큐 역할을 수행한다.
- 워커 풀 패턴: `net.Listen` 으로 수많은 연결이 들어올 때 고루틴을 무제한 생성하지 않고, 버퍼 채널을 이용해 임시 대기열(Job Queue)을 만든 뒤 정해진 개수의 워커 고루틴이 이를 나누어 처리하게 하여 서버 자원을 보호한다.

B. 뮤텍스

- 주요 용도: 단순 공유 자원(전역 변수, 카운터, 공유 맵 등)의 데이터 무결성 보호
- 작동 특징: `sync.Mutex` 또는 `sync/atomic` 을 사용하여 특정 메모리 영역에 한 번에 하나의 고루틴만 접근할 수 있도록 잠금(Lock)을 걸어 상호 배제를 달성한다.
- 선택 기준: 고루틴 간에 복잡한 데이터 흐름을 주고받을 필요 없이, 단순한 “동시 접근으로 인한 값 오염 방지”가 목적이려면 채널보다 뮤텍스를 사용하는 것이 코드 가독성과 미세한 성능(오버헤드 감소) 측면에서 유리하다.

(2) Go 런타임의 M:N 스케줄링(GMP 모델)

Go 언어가 수만 개의 동시 요청을 효율적으로 처리할 수 있는 핵심 이유는 OS 스레드에 의존하지 않고, 유저 레벨에서 동작하는 자체 런타임 스케줄러(GMP 모델)를 가지기 때문이다.

GMP 구조의 구성 요소

- G(Goroutine): 고루틴 그 자체를 의미한다. 컨텍스트 스위칭 비용이 수 킬로바이트(KB) 수준으로 매우 가볍고, 필요에 따라 스택 크기가 동적으로 늘어난다.
- M(Machine): 실제 운영체제(OS)의 커널 스레드를 의미한다. 실질적으로 CPU 위에서 명령어를 실행하는 주체이다.
- P(Processor) 논리 프로세서로, Go 코드를 실행하기 위한 자원과 컨텍스트를 가진다. 일반적으로 CPU 코어 개수만큼 생성되며, M 이 G 를 실행하기 위해서 반드시 P 를 할당받아야한다. P 는 실행 대기 중이고 고루틴들이 쌓여 있는 ‘로컬 실행 큐’를 관리한다.

M:N 스케줄링의 효율성

하나의 OS 스레드(M)가 멈추더라도 런타임은 P를 다른 스레드로 옮겨 실행 대기 중인 고루틴들을 계속 처리한다. OS 레벨의 무거운 스레드 스위칭을 피하고, 유저 레벨에서 가벼운 고루틴 스위칭을 수행하므로 대규모 동시성 제어에서 극도의 효율성을 보여준다.

(3) 네트워크 폴러(Network Poller)와 비동기 I/O

GMP 모델이 네트워크 서버 환경에서 진정한 성능을 발휘하게 만드는 숨은 공신은 Go 런타임 내부의 네트워크 폴러이다.

동작 메커니즘

1. I/O 블로킹 발생: 하나의 고루틴(G)이 `conn.Read()` 등을 호출하여 클라이언트의 데이터를 기다려야 하는 상황이 발생
2. 고루틴 분리: Go 런타임은 이 고루틴이 OS 스레드(M)을 붙잡고 멈춰 있게(Block) 두지 않는다. 해당 고루틴을 스레드에서 떼어내어 내부의 '네트워크 폴러' 영역으로 이동시킨다.
3. 스레드 재사용: 고루틴이 빠져나간 OS 스레드(M)는 쉬지 않고 로컬 실행 큐에 대기 중이던 다른 고루틴을 할당받아 즉시 다음 작업을 수행한다.
4. OS 이벤트 알림: 클라이언트로부터 데이터가 도착하면, OS 계층(Linux의 `epoll`, macOS의 `kqueue` 등)이 Go 런타임에 신호를 보낸다.
5. 작업 재개: 네트워크 폴러에서 대기하던 고루틴은 데이터가 준비되었음을 확인받고 다시 실행 대기 큐로 복귀하여 멈췄던 지점부터 코드를 이어나간다.

개발자는 코드를 작성할 때 복잡한 비동기 콜백 구조를 고민할 필요 없이, 마치 순환적으로 멈췄다 가는 '동기적 코드'형태로 편하게 작성할 수 있다.

하지만 Go 런타임 내부에서는 네트워크 폴러와 GMP 스케줄러가 결합하여 시스템 자원을 유기적으로 회전시키므로, 백엔드는 완전한 '비동기,논블로킹'의 최고 효율로 동작하게 된다.

[11] 코드 설명

[11-1] 전역 변수로 선언된 `type client chan <- string`

(1) `type` 키워드를 통한 타입 별칭

Go에서는 기존에 존재하는 타입을 기반으로 새로운 고유 타입을 만들 수 있다.

- 구조: `type [새로운 타입 이름] [기반이 되는 기존 타입]`

예를 들어, `type score int` 라고 쓰면 앞으로 `score` 라는 단어를 `int` 처럼 쓸 수 있는 것과 같다.

(2) `chan <- string`(단방향 송신 채널)

화살표(<-)의 위치가 핵심이다. 채널 키워드(`chan`) 오른쪽에 화살표가 붙으면 “채널 안으로 데이터를 밀어 넣기만 하겠다”는 뜻이다.

- `chan string`: 읽고 쓰기 모두 가능한 일반 채널(양방향)
- `chan <- string`: 데이터를 보내기만(Send)가능한 채널(단방향)
- `<- chan string`: 데이터를 받기만(Receive) 가능한 채널(단방향)

즉, 이 채널은 `ch <- "hello"`처럼 문자열을 보낼 수는 있지만, `msg := <-ch`처럼 문자열을 꺼내려고 하면 컴파일 에러가 발생한다.

(3) 왜 전역변수 구역에 이렇게 선언할까?(의도와 목적)

이 문법은 주로 채팅서버나 브로드캐스트(Broadcast) 시스템을 만들 때 자주 사용된다. 서버가 여러 클라이언트에게 메시지를 뿌려줘야 할 때, 서버 입장에서는 클라이언트의 채널에 메시지를 보내기만 하면 된다.

클라이언트 채널에서 서버가 뭔가를 읽을 필요는 없기 때문이다.

따라서 코드를 작성할 때 실수로라도 클라이언트 채널에서 데이터를 읽는 버그를 방지하기 위해, “이 타입은 안전하게 송신만 가능한 채널이야”라고 명시적으로 제약을 걸어둔 것이다.

```
package main

// 1. 단방향 송신 채널 타입을 client 라는 이름으로 정의
type client chan <- string

// 2. 여러 클라이언트를 관리하는 전역 맵(예시)
var entering = make(map[client]bool)

func main() {
    // 실제 채널을 만들 때는 양방향으로 만든다.
    ch := make(chan string)
    // client 타입 변수에 대입하여 자동으로 '송신 전용'으로 형변환
    var c client = ch
    // 데이터 송신은 가능
```

```
c <- "Welcome"
}
```

“문자열을 보내기만 할 수 있는 채널”에 client 라는 직관적인 이름을 붙여서, 코드의 안전성과 가독성을 높이기 위해 사용하는 문법이다.

[11-2] 전형적인 Go 기반 멀티유저 채팅 서버의 핵심 제어 통로 구축

```
var (
    entering = make(chan client)
    leaving  = make(chan client)
    messages = make(chan string) // 모든 클라이언트가 보내는 메시지 공용 통로
)
```

이 코드의 핵심은 “채널의 데이터 타입으로 또 다른 채널(client)을 넘겨주고 있다”는 점이다. 이를 Go에서는 채널의 채널(Channel of Channels)이라고 부른다.